

Multi-Cloud Wildlife-Image Job-Processing Service

Design Report

Lucas Huang

9 October 2026

Summary. A two-provider design (AWS + GCP, Sydney) for partner-submitted image batches processed by a CPU → GPU → CPU pipeline within a 20-VM / 4-GPU cap. Workers pull fenced leases from a single transactional job store; outputs are create-only per attempt, and a conditional commit publishes exactly one result per stage. Job metadata is replicated *synchronously* between the clouds and governed by a three-voter quorum (AWS, GCP, third-site witness), so failover is automatic, an isolated old primary cannot acknowledge a write, and no acknowledged decision is lost. Bulk data stays in its home cloud to avoid egress.

Contents

1	Overview, Assumptions and Key Decisions	1
2	One Job’s Lifecycle	2
3	Job Coordination and Placement	2
4	VMs, Containers and Orchestration	2
5	Storage and ML Reproducibility	3
6	Health and Recovery	3
7	Failure Scenario: GPU VM Lost After Writing Its Output	3
8	Multi-Cloud Operation	3
9	Cloud Security	4
10	Trade-offs, and Lessons from SmartPark	5

List of Figures

1	Architecture. Red numbers follow one job; teal links are the only cross-cloud traffic.	1
---	--	---

List of Tables

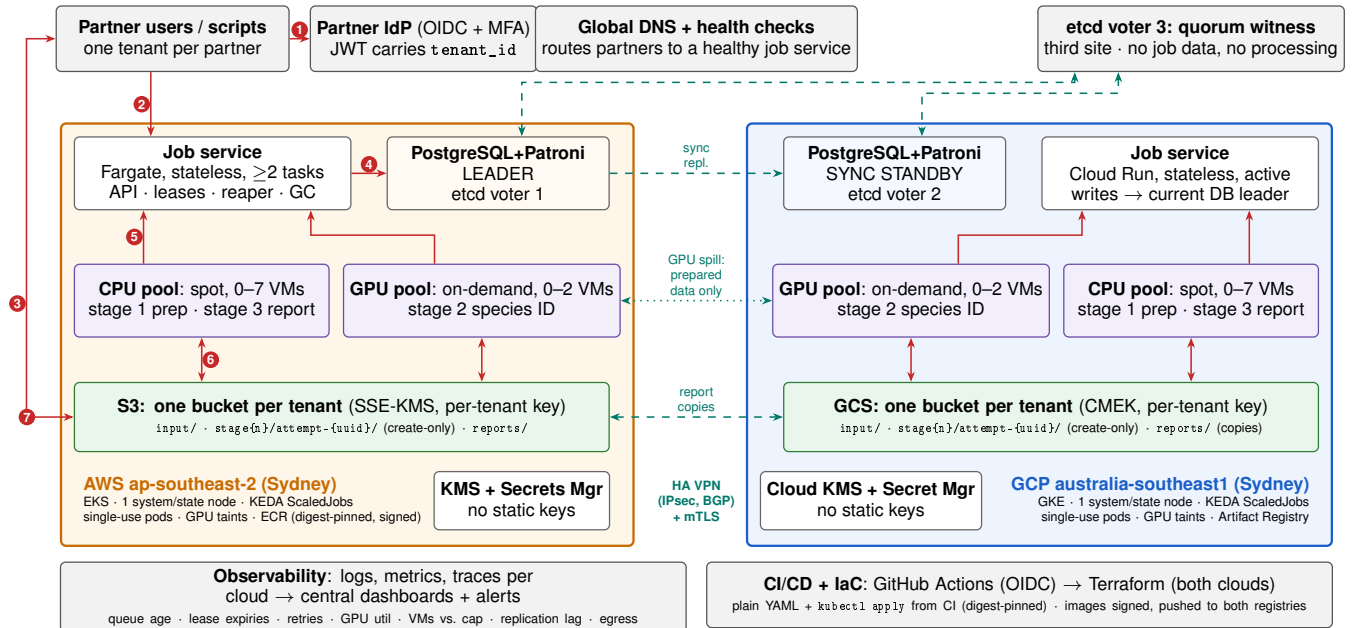
1	Assumptions and their design consequences.	1
2	Storage choices.	3
3	Recovery timeline for the required failure scenario.	4
4	Behaviour under provider and network failures.	4
5	Key decisions and trade-offs.	5
6	Lessons carried over from SmartPark.	5

1 Overview, Assumptions and Key Decisions

Partners upload a 1–5 GB image batch; each job runs *prep* (CPU) → *species ID* (GPU) → *report* (CPU). Four principles drive the design. (i) One transactional store (PostgreSQL) holds job state, the queue and leases, so there is never a dual-write between a queue and a database. (ii) *Execution is at-least-once, publication is exactly-once*: workers pull fenced leases, write create-only per-attempt objects, and a conditional update publishes exactly one attempt. (iii) *Compute is active-active across AWS and GCP*, and jobs run data-locally to avoid egress. (iv) *State is quorum-governed*: every commit is flushed synchronously in both clouds, and a three-voter quorum (AWS, GCP, third-site witness) decides which database may accept writes. Two sites alone cannot tell a dead peer from a broken link; a third voter can [1, 4].

Assumption	Value and consequence
Workload	100 jobs/day; burst 100/h; mean batch 3 GB ($\approx 1,000$ JPEGs); mean job 15 min split 5/6/4 min (prep/ID/report); prep output ≈ 5 –10% of input; report ≈ 10 MB. Sizing hypotheses, to be benchmarked on each VM type.
GPU is the bottleneck	4 GPUs $\times 60/6 = 40$ jobs/h. A 100-job burst needs ≥ 2.5 h of GPU service plus pipeline fill and cold start; ≈ 60 jobs still wait at the end of the burst hour $\Rightarrow$ durable queue and ETA; acceptance never blocks on capacity. Daily GPU use ≈ 10 of 96 GPU-h ($\approx 10\%$) $\Rightarrow$ GPU pools scale 0–2 per cloud.
CPU sizing	Burst prep 100×5 min ≈ 8.3 VMs; report is GPU-bound (40×4 min ≈ 2.7 VMs) $\Rightarrow \approx 11$ of 14 CPU workers at peak, if placement is balanced.
VM cap	Per cloud: 1 on-demand <i>system/state</i> node (KEDA, CoreDNS, PostgreSQL + Patroni, etcd voter) + 0–7 CPU + 0–2 GPU = 10, so 2 + 14 + 4 = 20 . Managed services (EKS/GKE masters, Fargate/Cloud Run, object storage) are outside the cap.
Network	Partner $\rightarrow$ storage burst 300 GB/h ≈ 670 Mbps (1.1 Gbps if every batch is 5 GB): uploads go directly to storage. AWS $\leftrightarrow$ GCP: 4 IPsec tunnels, one flow capped at ≈ 1.25 Gbps [16]; plan on 1 Gbps effective. Same-metro RTT < 5 ms, so a synchronous commit costs only a few ms.
Pricing (planning figures, not quotes)	Egress \$0.12–0.19/GB by provider and path [22, 23]; GPU VM $\approx \$1.2$ /h; object storage $\approx \$0.025$ /GB-month. Always-on 4 GPUs $\approx \$3.5$ k/month vs. on demand (≈ 10 GPU-h/day + 1 warm GPU in business hours) $\approx \$0.8$ k/month, for a ≈ 5 min cold start (to be measured).
Regions	Both clouds in Sydney: widest Australian GPU supply; data stays in Australia, because camera traps also photograph people [25]. GPU models differ by provider and region [21], so each pool uses what its quota offers, and the hardware is recorded in provenance.
Third parties	Partner OIDC IdP; provider-neutral DNS; GitHub Actions; and the <i>quorum witness</i> : one etcd voter at an independent third site, holding only leader keys (no job data, no processing, so not a third compute provider). Images and model weights are mirrored into our own registries and buckets.

Table 1: Assumptions and their design consequences.



Job lifecycle: (1) log in; (2) POST /jobs, routed by DNS to a healthy job service; (3) multipart upload straight to presigned URLs in the home cloud; (4) on submit: SHA-256 verified, ACCEPTED committed synchronously in *both* clouds, 202; (5) runners pull a lease for their pool; (6) read the published input, write a create-only attempt prefix; the coordinator publishes it with a fenced conditional commit; (7) report verified in the home cloud, COMPLETED, second copy added by a tracked copier; download via a presigned URL. **Teal:** cross-cloud traffic. Patroni elects one DB leader through a 3-voter etcd quorum (AWS, GCP, witness); every commit is flushed in both clouds. VM cap: 2 system/state + 14 CPU + 4 GPU = 20.

Figure 1: Architecture. Red numbers follow one job; teal links are the only cross-cloud traffic.

2 One Job’s Lifecycle

States. Job: UPLOADING → ACCEPTED → RUNNING → COMPLETED; side exits FAILED_NEEDS_REVIEW, CANCELLED. Each stage: QUEUED → RUNNING → SUCCEEDED, with RETRY_WAIT and FAILED. Job and attempt IDs are UUIDs. (1) The partner signs in through its IdP; the job service validates the JWT (issuer, audience, signature, expiry) and maps the partner to a tenant server-side. (2) POST /jobs (file list, sizes, SHA-256) picks the home cloud (healthy, earliest estimated finish) and returns per-object multipart-upload URLs scoped to jobs/{id}/input/ in the tenant bucket (15 min, renewable after re-authentication). (3) The client uploads directly; failed parts are retried individually, and abandoned uploads expire by lifecycle rule. (4) POST /jobs/{id}:submit carries a tenant-scoped Idempotency-Key and request hash. The service verifies sizes and SHA-256 (multipart ETags are not content hashes), then one transaction records ACCEPTED, the immutable input manifest, pinned image and model versions and PREP_QUEUED. Because commits are synchronous, 202 means the acceptance is durable in *both* clouds. A retry returns the same job; reusing a key with a different request is rejected. (5) A ready CPU runner in the home cloud claims the prep lease. (6) It writes stage1/attempt-{uuid}/; the coordinator publishes it (§7), marking the stage SUCCEEDED and queuing stage 2 in the same transaction. Stage 3 follows the same way. (7) The report is verified in the home cloud; one transaction records COMPLETED, the object version and a notification outbox event, and a tracked copier adds the second copy in the other cloud. Notifications are at-least-once with a stable event ID. GET /jobs/{id} shows stage, queue position and ETA; reports download via 5-minute presigned URLs.

3 Job Coordination and Placement

Acceptance and tracking. The jobs and stage_runs tables are the single source of truth. Each stage row holds status, epoch, attempt_id, owner, lease_expires_at (database clock) and published_attempt. Load is <1 write/s, so a separate broker (SQS/Pub/Sub) would only add a dual-write hazard.

Worker discovery and assignment (pull). A small *runner agent* in each stage image authenticates with workload identity and calls POST /leases with its capabilities (cloud, pool, allocatable memory, GPU model and VRAM, readiness), checked against the pool’s configured identity. The coordinator selects an eligible row with SELECT ...FOR UPDATE SKIP LOCKED [8] and, in the same transaction, sets RUNNING, increments epoch, creates a new attempt_id, records the owner and grants a 90s lease, renewed every 30s. Only a claim increments the epoch. No registry is needed: a worker exists while it holds or requests leases, and idle (faster) runners ask sooner, which balances load naturally.

Dependencies, resources and fairness. A stage becomes QUEUED only in the transaction that marks its predecessor SUCCEEDED; consumers require SUCCEEDED and read only the published manifest. Each job gets a resource profile from image count and decoded size (batch bytes alone do not predict peak memory) plus model VRAM; a lease matches only a compatible pool, and GPU stages run only on tainted GPU pools. Ordering is weighted fair share across tenants, then oldest first, with a per-tenant GPU cap (default 2) whose unused share may be borrowed.

Placement. Work is placed by estimated finish time (queue wait + measured runtime + transfer). Prep runs in the home cloud. Stage 2 *spills* only if $ETA_{remote} + t_{transfer} + 20 \text{ min} < ETA_{local}$, copying the verified *prepared* output ($\approx 150\text{--}300 \text{ MB}$) instead of the raw 3 GB batch: 10–20× less egress. The report stage stays next to the identification output.

4 VMs, Containers and Orchestration

Provisioning (IaC). Terraform modules per cloud (VPC, VPN/BGP, EKS/GKE, node pools, buckets with lifecycle rules, KMS, IAM) plus a global stack (DNS, witness). State is encrypted and locked. GitHub Actions runs **plan** on pull requests and **apply** after review, using OIDC federation, so CI holds no cloud keys. Kubernetes manifests are *plain YAML* (shared base plus small per-cloud files), extending my A1 SmartPark **kubect1** workflow [26]: CI substitutes signed image digests and runs **kubect1 apply** on both clusters, and a scheduled **kubect1 diff** reports drift. PostgreSQL + Patroni runs as a StatefulSet pinned to each cloud’s system/state node, on an encrypted SSD volume with the Linux watchdog enabled [6]. Failover drills run with each release.

Holding the cap. Node pools have max_size system 1, CPU 7 and GPU 2 per cloud. Upgrades and repairs must not exceed it, so a node is removed before its replacement is created (EKS MINIMAL updates, GKE zero-surge) [17, 18]. Restricted provisioning roles stop pools being added outside Terraform; an alert fires at 19 VMs. An *unreachable* VM still counts until its termination is confirmed. CPU workers use spot (a preemption is just a lease expiry); GPU and system/state nodes are on-demand.

Containers. One image per stage: the supplied program, the runner agent and pinned libraries, built once, scanned, signed with cosign [24], pushed to both registries and deployed *by digest*. The GPU image pins CUDA, framework and a tested driver range. As in A1, model weights are not baked into the image: an init container fetches **model@version**, verifies its SHA-256 and keeps it in a node-local, read-only, model-only cache; tenant data never enters it.

Isolation and scaling. Each attempt is a finite Kubernetes Job: KEDA *ScaledJobs* [11] launch one single-use pod per eligible queued lease, counting pending pods to avoid over-launch. A pod claims a lease only after its startup checks pass, runs one attempt and exits, so every tenant job gets a fresh pod and scratch **emptyDir**. Requests cover CPU, RAM, ephemeral disk and GPU, and required anti-affinity places one attempt per worker VM. The cluster autoscaler works within pool bounds; GPU pools scale to zero after 10 min idle (one warm in business hours). Pods

run non-root with a read-only root filesystem, no capabilities and seccomp. Default-deny NetworkPolicy allows egress only to the coordinator, identity, DNS, model and storage endpoints, sharply limiting exfiltration by third-party code.

5 Storage and ML Reproducibility

Data	Store	Durability, lifecycle and cleanup
Inputs	S3/GCS, bucket + key per tenant, home cloud	Versioned; deleted 30 days after the job ends (longer under review); partners keep originals. Not replicated cross-cloud (≈ 9 TB/month), so a home-storage outage pauses those jobs.
Intermediates	same bucket, <code>stage{n}/attempt-{uuid}/</code>	<i>Create-only</i> writes (S3 <code>If-None-Match</code> , GCS <code>ifGenerationMatch=0</code>) [13, 14]; <code>_MANIFEST.json</code> (object versions + SHA-256) written last. GC checks DB references and leases; unpublished attempts go after 24 h, published ones 7 days after the job ends.
Reports	home cloud + copy in the other	Verified before <code>COMPLETED</code> ; second copy tracked until verified; versioned; workers cannot delete; kept 1 year by policy, Infrequent Access after 30 days.
Metadata	PostgreSQL + Patroni, one node per cloud	Leader + <i>synchronous</i> standby (every commit in two clouds); continuous WAL archiving to S3 and GCS, PITR 14 days, restores tested.

Table 2: Storage choices.

Provenance. Each published stage run stores an immutable record: input/output manifest hashes, attempt and epoch, image digest, program version, model name/version/SHA-256, framework/CUDA/driver versions, GPU model, instance type, region, timings and exit code. Versions are pinned at acceptance and the report embeds a provenance digest. Floating point can differ slightly across GPU models, so hardware is recorded and pinned when bitwise reproducibility is required. Referenced images and models are kept as long as the reports.

6 Health and Recovery

Readiness. A startup check verifies the model checksum, GPU allocation and a CUDA self-test, scratch space, and storage and coordinator reachability before any claim. Node health comes from the kubelet and GPU (Xid) errors; DB health from Patroni’s REST checks and etcd quorum status. **Crashes.** A non-zero exit or OOM is reported by the agent; a lost pod, VM, preemption or partition appears as missing heartbeats. The reaper locks the row, confirms expiry on the database clock, marks the attempt `ABANDONED` and requeues; the next claim increments the epoch. **Stalls.** Heartbeats carry progress (output bytes, program progress, GPU utilisation for stage 2; an idle CPU does not mean a stalled GPU kernel). No progress for 10 min, or exceeding $2\times$ the p99 for the size class and hardware, kills and requeues the attempt. The agent stops itself when it cannot renew, but safety never depends on that (§7).

Repeated failures are classified: (a) transient program failures get at most two retries with exponential backoff and jitter, and an OOM moves to a larger pool; (b) deterministic input or configuration errors are not retried and are returned to the partner with a reason; (c) infrastructure failures have a separate budget (5 attempts or 24 h) and lower the node’s health score, and 2 failures from different jobs within 1 h quarantine the node. Budgets are initial defaults, tuned by fault injection. An exhausted budget leaves the job `FAILED_NEEDS_REVIEW` with data kept and an alert raised.

Logs and metrics. Structured logs carry `tenant`, `job`, `stage`, `attempt`, `epoch`, `trace ID`, `image digest`, `model hash` and failure category, never images or secrets. Alerts: oldest queue age, stage p50/p99 by hardware, GPU utilisation/VRAM, lease-expiry and retry rates, tenant fairness, sync-standby state, etcd quorum, report-copy backlog, VM count vs. cap and egress bytes.

7 Failure Scenario: GPU VM Lost After Writing Its Output

Only the coordinator publishes, and only through a fenced conditional commit; Table 3 traces the required scenario step by step. **Why there is no duplicate or conflicting result.** (a) Attempts never share a path and writes are create-only, so G1 cannot overwrite G2. (b) Exactly one attempt can satisfy the predicate (a fencing token checked by the store itself [2, 3]), and cancellation takes the same row lock. (c) Consumers require `SUCCEEDED` plus the pointer, so an orphan is invisible. (d) `COMPLETED` and the outbox event share one transaction; notifications carry a stable ID. (e) Leases use the database clock, and because leases live in the synchronously replicated store, the same rules hold after a DB failover. The guarantee is one *authoritative published result*, not one physical execution. *Considered and rejected:* adopting G1’s verified orphan saves ≈ 6 GPU-min but adds a second publication path.

8 Multi-Cloud Operation

Strategy: active-active compute, quorum-governed state. Both clouds run job services and workers every day. PostgreSQL runs one Patroni-managed node per cloud: the *leader* (normally AWS) and a *synchronous standby* (GCP) with `synchronous_commit=on`, so a commit is acknowledged only after it is flushed in both clouds [7]. Patroni keeps its leader key in a three-voter etcd cluster (AWS, GCP, witness) [5]. Job services in both clouds write to the current

Time	Event and state change
t_0	G1 has claimed stage 2 of job J: epoch=7 , attempt A. It writes stage2/attempt-A/ with create-only writes, then a manifest of object versions and SHA-256. Only the coordinator can move the publication pointer.
t_1	G1 becomes unreachable before committing; heartbeats stop. Its storage credentials may outlive the lease (AWS STS sessions last ≥ 15 min [12]) but can write only attempt-A/ , which nobody reads unless published, so credentials are <i>not</i> the safety mechanism.
t_1+90 s	The reaper locks the row, confirms expiry, marks A ABANDONED and sets QUEUED; epoch stays 7. A commit from G1 now fails, because the stage is not RUNNING.
t_2	G2 claims (epoch=8 , attempt B), writes attempt-B/ and commits in one short transaction: lock the row, then <code>UPDATE stage_runs SET status='SUCCEEDED', published_attempt='B' WHERE job=J AND stage=2 AND attempt_id='B' AND owner='G2' AND epoch=8 AND status='RUNNING' AND lease_expires_at > clock_timestamp() → 1 row</code> , and queue stage 3. <code>clock_timestamp()</code> is used because <code>now()</code> is fixed at transaction start [9]. A repeated identical commit returns the recorded success.
t_3	G1 returns and commits with epoch=7 , attempt A $\rightarrow$ 0 rows $\rightarrow$ 409 STALE_LEASE. The agent discards its work; Kubernetes has already deleted the pod, so the kubelet kills it on reconnect.
t_3+24 h	GC confirms that A is unreferenced and deletes attempt-A/ .

Table 3: Recovery timeline for the required failure scenario.

leader through a Patroni-aware endpoint.

Connectivity and egress. AWS Site-to-Site VPN $\leftrightarrow$ GCP HA VPN (4 tunnels, BGP, non-overlapping CIDRs) [15], with mTLS for services, etcd peers and replication. Cross-cloud traffic is DB replication, writes from the GCP job service, report copies and rare spills; partners’ presigned-URL traffic never crosses clouds. Replicating every raw input would move ≈ 9 TB/month ($\approx \$1,080$ – $1,710$); data-local placement leaves reports (100×10 MB $\times 30 \approx 30$ GB/month) plus small DB/WAL traffic and occasional spills ($\approx \$0.02$ – 0.06 each).

Why failover is safe and automatic. (1) *An isolated old leader cannot acknowledge anything:* its commits wait for the unreachable standby, and switching to asynchronous mode requires updating the `/sync` key in etcd, which needs a quorum it no longer has. Once its leader key expires (TTL 30s) it demotes itself; the watchdog resets the node if Patroni itself hangs. (2) *Only an up-to-date node can be promoted:* Patroni promotes only members listed in `/sync`, and a member is removed from that list (by quorum) *before* the leader stops waiting for it. (3) *Degraded mode is explicit:* the surviving leader keeps writing asynchronously only after recording in etcd that the other node is ineligible; that node must catch up (via `pg_rewind`) before it can rejoin the list. Hence no acknowledged decision is ever lost or forked, and no human is needed.

Failure	Behaviour
AWS unreachable	GCP + witness hold quorum; GCP (in <code>/sync</code>) is promoted in ≈ 30 – 60 s with every acknowledged commit (RPO = 0), drops AWS from <code>/sync</code> and continues: admission, claims and publication run in GCP. DNS moves partner traffic in ≈ 3 min. AWS-homed inputs wait for AWS.
GCP unreachable	AWS + witness hold quorum; AWS drops GCP from <code>/sync</code> and continues. GCP-homed jobs pause; second report copies queue.
Link-only partition	AWS keeps the leader key and continues; GCP cannot be promoted (no quorum, not in <code>/sync</code>). GCP runners finish current attempts; their commits are retried when the link returns.
Witness down	AWS + GCP still form a quorum: no effect beyond an alert.
Witness + one cloud	No quorum: the survivor serves reads only. An operator runbook (fence the other side via its control plane, verify, promote) is the last resort; it is a double failure.

Table 4: Behaviour under provider and network failures.

Capacity after losing a cloud is ≤ 10 VMs (2 GPU, 20 jobs/h), because unreachable VMs still count. **Failback:** the old leader is rewound and resynchronised as standby, then switched back in a quiet window.

9 Cloud Security

Users. Partners sign in through OIDC with MFA; issuer, audience, signature and expiry are validated and the tenant is mapped server-side. Automation uses scoped OAuth client credentials. **Services.** Workload identity (EKS Pod Identity, GKE Workload Identity) [19, 20] means no static cloud keys. Internal APIs, PostgreSQL, Patroni REST and etcd use mTLS with per-service certificates. **Access control (defence in depth).** (1) The tenant comes from the token, never the request body; ownership is checked on every call. (2) Row-level security [10] via `SET LOCAL app.tenant_id` per transaction, with application roles that are not table owners and lack `BYPASSRLS`, and a separate restricted coordinator role. (3) One bucket and one KMS key per tenant per cloud. (4) Runners get only the exact input/model reads and attempt-prefix writes they need, and cannot publish or delete reports; since STS sessions last ≥ 15 min, the lease, not the credential, is the authority. (5) Presigned URLs are per object and short-lived. Retiring a tenant key crypto-shreds its data once retention duties are met. **Secrets** live in AWS Secrets Manager / GCP Secret

Manager, are read at run time and rotated; CI runs secret scanning. **Network isolation:** workers and databases have no public IPs and use private endpoints; the API sits behind a WAF and rate limits; admission control admits only signed images. **Encryption:** TLS 1.2+ externally, mTLS internally, IPsec between clouds; SSE-KMS/CMEK per tenant and encrypted volumes and backups at rest. Audit: CloudTrail data events, GCS access logs, DB audit of every state transition.

10 Trade-offs, and Lessons from SmartPark

Decision	Rejected alternative	Why	Accepted cost
PostgreSQL as job store and queue	SQS / Pub/Sub + DB	One transaction for state, queue and leases: no dual-write	The DB is critical, hence quorum replication
Pull-based fenced leases	Central push scheduler	Discovery and load balancing for free; the store enforces the fence	Polling delay of seconds
Patroni, synchronous standby, 3-voter etcd	RDS + async cross-cloud replica	Automatic failover, RPO = 0, isolated leader cannot acknowledge	Self-managed DB, +few ms per commit, third-site witness (fallback: operator-run promotion)
Data-local jobs, inputs not replicated	Mirror every batch	Saves $\approx$ \$1.1–1.7k/month of egress	Home-storage outage pauses those jobs
Spill prepared data only	Spill raw batch	10–20 $\times$ less egress per spill	Spill only after prep is published
Rerun after uncertain failure	Adopt the orphan output	Single publication path	$\approx$ 6 GPU-min per incident
GPUs scale to zero (+1 warm)	Always-on GPUs	Saves $\approx$ \$2.7k/month	$\approx$ 5 min cold start
Spot CPU, on-demand GPU	All on-demand / all spot	Preemption is just a lease expiry; GPU slots are scarce	Occasional CPU reruns

Table 5: Key decisions and trade-offs.

What SmartPark (A1) actually ran [26]	Here	Reason
HPA scaling on CPU utilisation	KEDA scaling on queue depth	CPU cannot see a GPU backlog; queue depth can
Long-running Deployment behind a LoadBalancer	Finite single-use Jobs that pull work	5–30 min batch stages need per-tenant isolation and exactly-once publication
Init container loading the model at start	Kept, now pinned by version and SHA-256	Worked in practice; pinning makes results reproducible
Plain YAML applied with kubectl	Kept, run from CI on both clusters	Enough for two clusters; Helm/Argo CD would add controllers for little gain

Table 6: Lessons carried over from SmartPark.

References

- [1] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
- [2] M. Kleppmann, “How to do distributed locking,” 2016. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>
- [3] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [4] etcd Authors, “Frequently asked questions (cluster size and fault tolerance).” <https://etcd.io/docs/v3.5/faq/>
- [5] Patroni, “Replication modes: synchronous mode.” https://patroni.readthedocs.io/en/latest/replication_modes.html
- [6] Patroni, “Watchdog support.” <https://patroni.readthedocs.io/en/latest/watchdog.html>
- [7] PostgreSQL Global Development Group, “Synchronous replication,” PostgreSQL documentation. <https://www.postgresql.org/docs/current/warm-standby.html>
- [8] PostgreSQL Global Development Group, “SELECT: the locking clause (FOR UPDATE, SKIP LOCKED).” <https://www.postgresql.org/docs/current/sql-select.html>
- [9] PostgreSQL Global Development Group, “Date/time functions: current date/time.” <https://www.postgresql.org/docs/current/functions-datetime.html>
- [10] PostgreSQL Global Development Group, “Row security policies.” <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [11] KEDA Authors, “Scaling jobs (ScaledJob).” <https://keda.sh/docs/latest/concepts/scaling-jobs/>
- [12] Amazon Web Services, “AssumeRole,” AWS STS API reference. https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html
- [13] Amazon Web Services, “Conditional writes,” Amazon S3 user guide. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/conditional-writes.html>
- [14] Google Cloud, “Request preconditions,” Cloud Storage documentation. <https://cloud.google.com/storage/docs/request-preconditions>
- [15] Google Cloud, “Create HA VPN connections between Google Cloud and AWS.” <https://cloud.google.com/network-connectivity/docs/vpn/tutorials/create-ha-vpn-connections-google-cloud-aws>
- [16] Amazon Web Services, “Site-to-Site VPN quotas.” <https://docs.aws.amazon.com/vpn/latest/s2svpn/vpn-limits.html>
- [17] Amazon Web Services, “Understand each phase of node updates,” Amazon EKS user guide. <https://docs.aws.amazon.com/eks/latest/userguide/managed-node-update-behavior.html>
- [18] Google Cloud, “Node pool upgrade strategies,” GKE documentation. <https://cloud.google.com/kubernetes-engine/docs/concepts/node-pool-upgrade-strategies>
- [19] Amazon Web Services, “EKS Pod Identity.” <https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html>
- [20] Google Cloud, “Workload Identity Federation for GKE.” <https://cloud.google.com/kubernetes-engine/docs/concepts/workload-identity>
- [21] Google Cloud, “GPU regions and zones availability.” <https://cloud.google.com/compute/docs/gpus/gpu-regions-zones>
- [22] Google Cloud, “Cloud Storage pricing.” <https://cloud.google.com/storage/pricing>
- [23] Amazon Web Services, “Amazon EC2 on-demand pricing: data transfer.” <https://aws.amazon.com/ec2/pricing/on-demand/>
- [24] Sigstore, “Cosign: signing containers.” <https://docs.sigstore.dev/>
- [25] Office of the Australian Information Commissioner, “The Privacy Act.” <https://www.oaic.gov.au/privacy/the-privacy-act>
- [26] L. Huang, “SmartPark: ML-based cloud parking platform,” FIT3184 Assignment 1 (FastAPI, Docker, GKE, HPA, init-container model loading), Monash University, 2026.